

Does a Notch Make a Non-Linearity?

An Empirical Evaluation of HeLU against ReLU and GELU

HeLU activation study

`helu-activation-research/` in `lukeabraham24777/unblock-me-game`

September 2026

Abstract

We evaluate a proposed activation function, HeLU, defined as $\text{HeLU}(x) = x$ everywhere except on the interval $0.5 \leq x \leq 0.6$, where $\text{HeLU}(x) = 0.9x$. The hypothesis under test is that networks trained with HeLU outperform networks trained with ReLU or GELU. We compare the three activations, plus an identity (purely linear) control, under identical architectures, optimisers, and seeds on one-dimensional function regression, two-dimensional non-linearly separable classification, MNIST and Fashion-MNIST with multilayer perceptrons, Fashion-MNIST with a small convolutional network, a depth sweep, and a mechanistic analysis of the trained networks. **The hypothesis is not supported.** On every task whose solution requires a non-linear decision function, HeLU networks perform substantially and statistically significantly worse than ReLU and GELU networks, and are statistically indistinguishable from the linear control. A least-squares linear fit explains a fraction 1.00000 of the variance of a trained HeLU network’s outputs (versus 0.957 for ReLU), and only 2.81% of hidden pre-activations fall inside the notch after training. HeLU is therefore best understood as an approximately linear map with a small, rarely visited perturbation, and it inherits the expressivity limits of deep linear networks. The one setting in which HeLU comes within a point of ReLU and GELU is the convolutional network, where max-pooling supplies the non-linearity that the activation does not, and the linear control comes equally close there. An ablation over fifteen variants of the notch (moved, deepened, or widened) shows that the deficit is a property of the notch design rather than of its particular constants. All code, raw results, and figures are released alongside this document.

1 Introduction

The choice of activation function sets the family of functions a neural network can represent and shapes the optimisation landscape. ReLU [6] and GELU [2] dominate current practice; the former is cheap and sparse, the latter is smooth and has become the default in transformer architectures. Proposals for new activations are common, and the standard of evidence is empirical: a new function is judged by whether models trained with it reach better accuracy or lower loss than the same models trained with the incumbents [8].

This report applies that standard to a proposed activation, which we refer to as HeLU. It is defined as

$$\text{HeLU}(x) = \begin{cases} 0.9x & \text{if } 0.5 \leq x \leq 0.6, \\ x & \text{otherwise.} \end{cases} \quad (1)$$

The claim we set out to verify is that HeLU is *superior* to ReLU and GELU, i.e. that networks trained with HeLU achieve better held-out performance under matched conditions. We fixed the following reading of the outcome before running the full experimental sweep: the hypothesis is

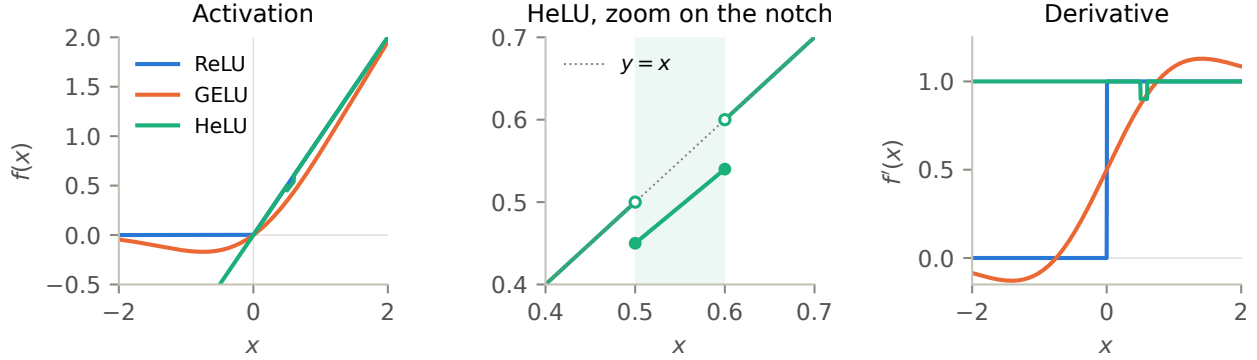


Figure 1: Left: ReLU, GELU, and HeLU on $[-2, 2]$. Centre: HeLU near the notch; filled and open circles mark the value taken and not taken at each edge. Right: derivatives. HeLU’s derivative is 1 except on $[0.5, 0.6]$, where it is 0.9.

supported if HeLU attains higher mean test accuracy (or lower test error) than both ReLU and GELU on the majority of tasks, with the difference significant at $p < 0.05$ under Welch’s t -test [10] across seeds.

Contributions.

1. A controlled comparison of HeLU, ReLU, and GELU, plus an identity control, over eight experiments spanning regression, classification, image benchmarks, and depth (Section 3).
2. A negative result: HeLU is statistically indistinguishable from a linear network on every task we tried and significantly worse than ReLU and GELU whenever the task is non-linear (Section 4).
3. A mechanistic explanation. The notch is visited by roughly 2.81% of pre-activations, and a linear model fits the trained HeLU network almost perfectly (Section 4.5).

2 The HeLU function

Figure 1 plots the three activations and their derivatives. Several properties of (1) follow directly from the definition.

- **It is almost the identity.** $|\text{HeLU}(x) - x| = 0.1|x| \leq 0.06$, and this deviation is non-zero only on an interval of width 0.1.
- **It is discontinuous.** At $x = 0.5$ the function jumps from 0.5 to 0.45; at $x = 0.6$ it jumps from 0.54 to 0.6. Neither ReLU nor GELU has a discontinuity.
- **Its derivative is nearly constant.** $\text{HeLU}'(x) = 1$ outside the notch and 0.9 inside; automatic differentiation reproduces exactly this (Figure 1, right). Gradients therefore flow through HeLU layers essentially unchanged, as in a linear network.
- **It is non-polynomial.** By the theorem of Leshno et al. [5], a single-hidden-layer network with HeLU activations is a universal approximator in the limit of infinite width. The theorem, however, says nothing about how many units, how large the weights, or how difficult the optimisation problem must be. For HeLU the non-linear budget of each unit is a perturbation of amplitude at most 0.06 confined to a band of width 0.1, so expressing a non-linear function of order-one amplitude requires either very many units whose pre-activations land in the notch or very large output weights.

The natural null model for HeLU is therefore a *deep linear network*, whose behaviour is well understood [9]: regardless of depth or width it can only represent a single affine map of its input. We include the identity activation as an explicit control so that this null can be tested directly rather than assumed.

3 Experimental setup

All experiments use PyTorch [7] on CPU with the Adam optimiser [3]. Within each experiment the architecture, learning rate, batch size, number of steps or epochs, data ordering, and parameter-initialisation seed are identical across activations; the activation module is the only thing that changes. Hyper-parameters were fixed in advance to common defaults and were not tuned per activation. Results are reported as mean $\pm$ sample standard deviation across seeds, and differences involving HeLU are tested with Welch’s unequal-variance t -test.

E1: Function shape. Analytic evaluation of f and f' (Figure 1).

E2: 1-D regression. An MLP with two hidden layers of 64 units maps $x \in [-2, 2]$ to one of four targets: $\sin 3x$, $|x|$, x^2 , and the step $\mathbf{1}[x > 0]$. Training uses 512 points with Gaussian noise ($\sigma = 0.05$), 3000 full-batch Adam steps at learning rate 3×10^{-3} ; the test MSE is computed on 2000 noise-free points. 5 seeds.

E3: 2-D classification. Two-moons and two-spirals datasets (600 training points, 4000 test points), same MLP with a two-way softmax output, 3000 steps. 5 seeds.

E4: MNIST and Fashion-MNIST, MLP. 784–256–256–10 MLP, batch size 128, learning rate 10^{-3} , 5 epochs. 5 seeds each [4, 11].

E5: Fashion-MNIST, CNN. Two 3×3 convolutions (32 and 64 channels), each followed by the activation and 2×2 max-pooling, then a 128-unit fully connected layer with the activation and a linear classifier. Same optimiser settings, 5 epochs, 3 seeds.

E6: Depth sweep. Fashion-MNIST MLP with 128-unit hidden layers and 1, 2, 4, or 8 hidden layers, 3 epochs, 3 seeds.

E7: Linearity and notch occupancy. For the Fashion-MNIST MLP we measure (i) the R^2 of the best least-squares affine map from the 784-dimensional input to the network’s 10 logits, computed on 2000 test images, and (ii) the fraction of hidden pre-activations lying in $[0.5, 0.6]$, both at initialisation and after 3 epochs of training. 3 seeds.

E8: Kernel cost. Wall time of forward plus backward through the activation alone on a 4×10^6 -element tensor, 20 repetitions.

E9: Notch variants. Fifteen variants of (1) obtained by moving the notch ($[0.5, 0.6]$, $[0.1, 0.2]$, $[-0.05, 0.05]$), deepening it (scale 0.9, 0.8, 0.5, 0.2), or widening it ($[0.5, 1.0]$, $[0.5, 2.0]$, $[0.5, \infty)$ at scale 0.2). Each variant is run under the E2 protocol on $\sin 3x$ and x^2 , the E3 protocol on moons and spirals, and the E4 Fashion-MNIST MLP protocol for 3 epochs, all with 3 seeds.

4 Results

4.1 Function approximation

Figure 2 and Table 1 summarise E2. ReLU and GELU networks fit all four targets closely, with test MSE of order 10^{-4} to 10^{-3} . The HeLU network does *not* collapse to the linear control’s straight line: with full-batch training on a one-dimensional input, gradient descent does find the notch and uses it. What it builds with it, however, is a jagged staircase. Because the only non-linear features of HeLU are its two jump discontinuities, the network can represent vertical jumps but not smooth curvature,

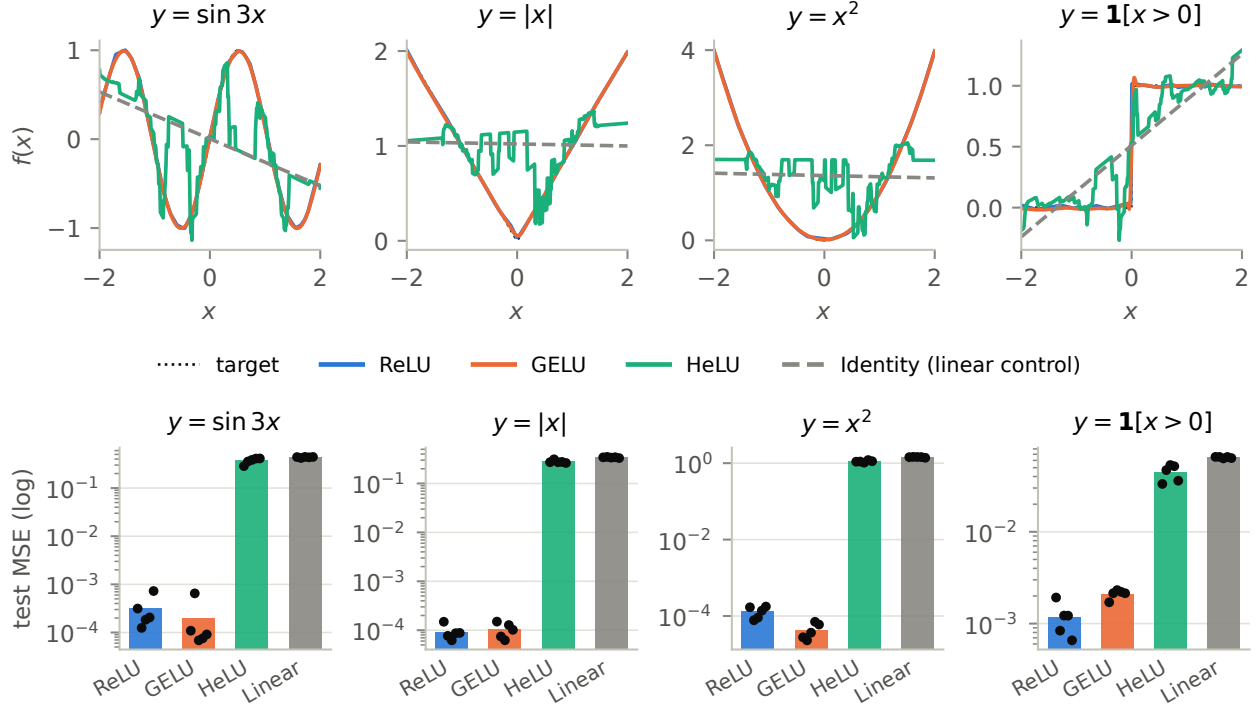


Figure 2: 1-D regression (E2). Top: the seed-0 fit of each network against the target (dotted). Bottom: test MSE on a log scale, bars are means over 5 seeds, dots are individual seeds. HeLU and the linear control coincide almost exactly.

Table 1: 1-D regression test MSE (mean $\pm$ sd over 5 seeds). Lower is better.

Target	ReLU	GELU	HeLU	Linear
$\sin(3x)$	0.0003 ± 0.0002	0.0002 ± 0.0003	0.3695 ± 0.0518	0.4364 ± 0.0093
$ x $	0.0001 ± 0.0000	0.0001 ± 0.0000	0.2792 ± 0.0193	0.3409 ± 0.0071
x^2	0.0001 ± 0.0000	0.0000 ± 0.0000	1.1113 ± 0.0634	1.4361 ± 0.0297
$\mathbf{1}[x > 0]$	0.0012 ± 0.0005	0.0021 ± 0.0002	0.0443 ± 0.0092	0.0646 ± 0.0012

and the fit consists of a linear trend decorated with irregular steps of amplitude comparable to the notch itself. The resulting test MSE is three orders of magnitude worse than ReLU or GELU on $\sin 3x$, $|x|$, and x^2 (Table 1), and only 15–25% better than the linear control. The step target is the most favourable case for HeLU, since a discontinuity is exactly what it can express, and even there its error is 30–40 $\times$ that of ReLU and GELU.

4.2 Non-linearly separable classification

Two-moons and two-spirals are the canonical tests of whether a model can learn a curved decision boundary. Figure 3 shows that ReLU and GELU do, reaching 98.7% and 98.4% on moons and 99.6% and 99.6% on spirals. HeLU reaches 88.0% on moons and 63.0% on spirals, against 87.7% and 64.9% for the linear control (Table 2). Its decision boundary is the linear control’s straight line with small, spiky perturbations where a hidden unit’s pre-activation crosses the notch, visible as faint striations in Figure 3. On moons these perturbations buy a fraction of a percentage point; on spirals they cost one, and HeLU is slightly *worse* than a linear classifier. In neither case does HeLU

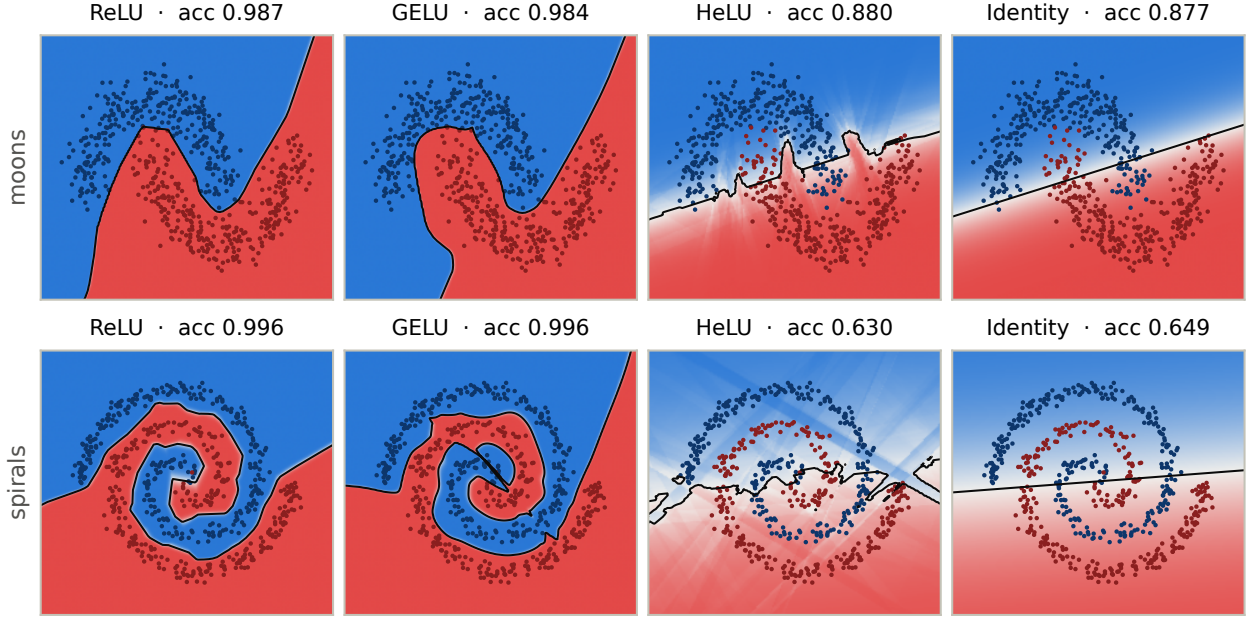


Figure 3: Decision functions on two-moons (top) and two-spirals (bottom), E3. Colour is the predicted probability of class 1; the black curve is the 0.5 level set. Titles give mean test accuracy over 5 seeds. HeLU produces the linear control’s straight boundary with small, spiky perturbations where hidden pre-activations cross the notch.

Table 2: 2-D classification test accuracy (mean $\pm$ sd over 5 seeds) and Welch p -values for the difference between HeLU and each non-linear baseline.

Dataset	Test accuracy				p (HeLU vs.)	
	ReLU	GELU	HeLU	Linear	ReLU	GELU
moons	0.9873 ± 0.0017	0.9845 ± 0.0016	0.8803 ± 0.0078	0.8769 ± 0.0032	$< 10^{-4}$	$< 10^{-4}$
spirals	0.9962 ± 0.0016	0.9963 ± 0.0024	0.6303 ± 0.0121	0.6486 ± 0.0061	$< 10^{-4}$	$< 10^{-4}$

approach the non-linear baselines, and both differences from ReLU and GELU are significant at $p < 10^{-4}$.

4.3 Image classification

Table 3 and Figures 4 and 5 report E4 and E5. On the MNIST MLP, ReLU and GELU reach 97.57% and 97.91% while HeLU reaches 91.50%, essentially the accuracy of a multinomial logistic regression, which is what a deep linear network trained with cross-entropy computes (linear control: 91.50%). On the Fashion-MNIST MLP the gap is similar: 88.14% and 88.16% against 83.33% (linear: 83.33%). In both cases the HeLU-versus-ReLU and HeLU-versus-GELU differences are significant at the $p < 0.05$ level, whereas HeLU versus the linear control is not. The training-loss curves in Figure 4 show that HeLU does not optimise faster in early training either: its loss tracks the linear control’s from the first steps and plateaus at the same, higher, level.

The convolutional network narrows the gap without closing it. Here HeLU reaches 90.35%, about one point below ReLU (91.48%, $p = 0.04$) and GELU (91.96%, $p = 0.03$). But the linear control also reaches 90.32%, and HeLU is again indistinguishable from it ($p = 0.93$). The reason the

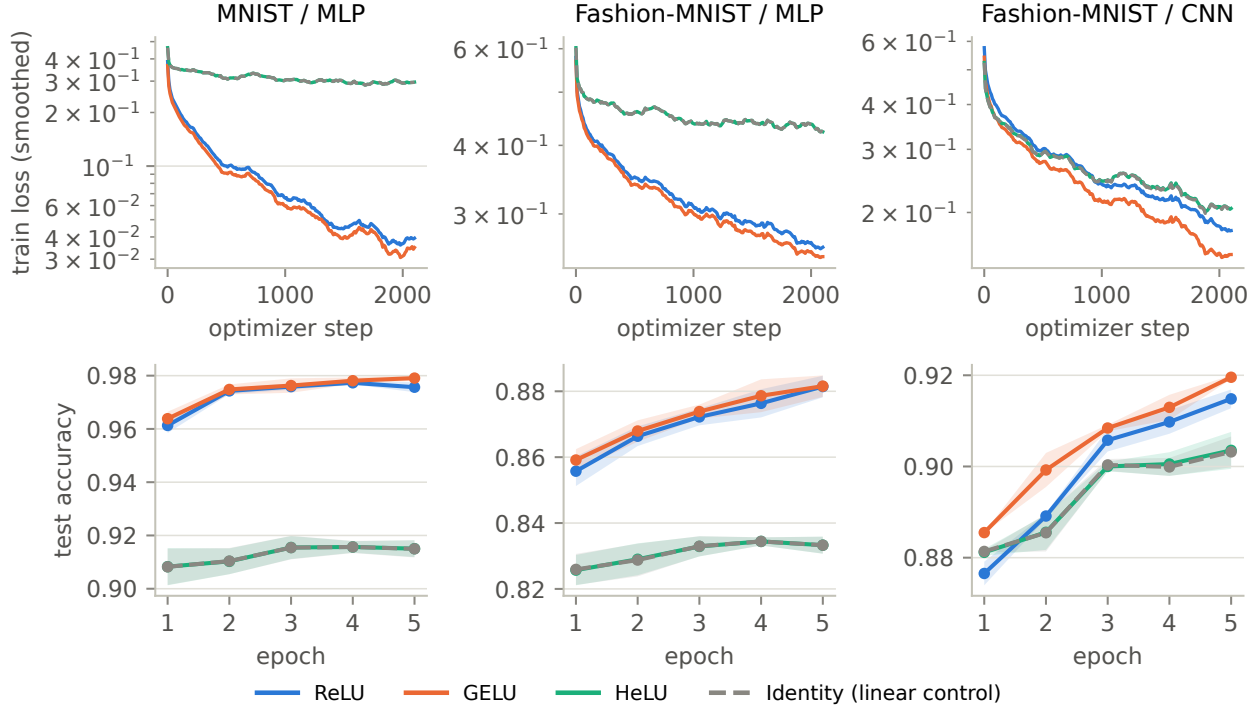


Figure 4: Training dynamics on the image benchmarks (E4, E5). Top: training loss, averaged over seeds after a 25-step moving average. Bottom: test accuracy after each epoch, shaded bands are ± 1 sd over seeds.

Table 3: Image benchmarks: final test accuracy (mean $\pm$ sd; 5 seeds for the MLPs, 3 for the CNN) and Welch p -values for HeLU against each other activation.

Benchmark	Test accuracy (mean $\pm$ sd)				p (HeLU vs.)		
	ReLU	GELU	HeLU	Linear	ReLU	GELU	Linear
MNIST, MLP	0.9757 $\pm$ 0.0021	0.9791 $\pm$ 0.0011	0.9150 $\pm$ 0.0035	0.9150 $\pm$ 0.0036	$< 10^{-4}$	$< 10^{-4}$	0.98
Fashion-MNIST, MLP	0.8814 $\pm$ 0.0037	0.8816 $\pm$ 0.0036	0.8333 $\pm$ 0.0029	0.8333 $\pm$ 0.0029	$< 10^{-4}$	$< 10^{-4}$	1
Fashion-MNIST, CNN	0.9148 $\pm$ 0.0025	0.9196 $\pm$ 0.0005	0.9035 $\pm$ 0.0050	0.9032 $\pm$ 0.0041	0.0395	0.0293	0.933

gap is small is that the architecture contains a second non-linearity, the 2×2 max-pooling after each convolution. Max-pooling alone is enough to make the convolutional network a competent non-linear classifier on this dataset, so the activation function’s contribution is largely masked. The CNN result is therefore mostly evidence about max-pooling, not about HeLU; a network with no activation at all obtains the same accuracy.

4.4 Depth

If the notch were being used as a non-linearity, stacking more HeLU layers would compound it and accuracy would change with depth in a way that differs from the linear control. Figure 6 and Table 4 show no such divergence: HeLU’s accuracy is flat from one to four hidden layers and degrades at eight, and it coincides with the linear control to within 0.05 percentage points at every depth. (ReLU and GELU also degrade slightly at depth 8 under this short, untuned 3-epoch budget, but they remain 3–4 points above the linear pair throughout.) A product of affine maps is an affine

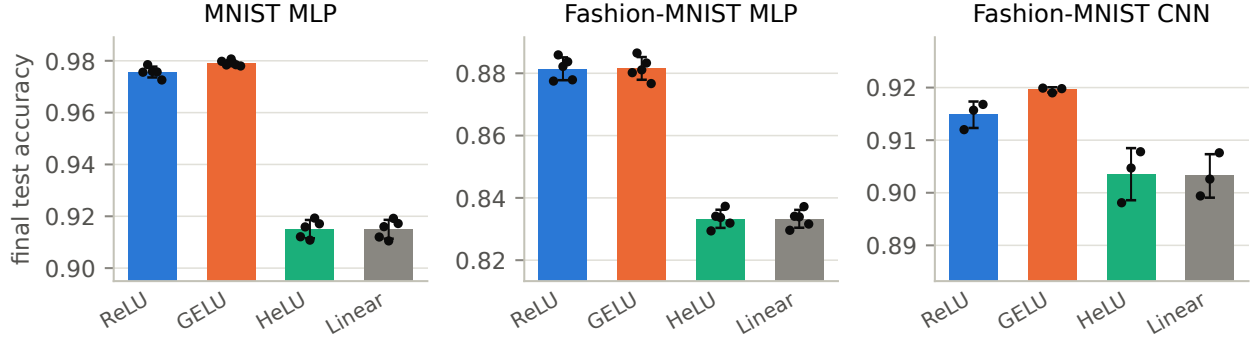


Figure 5: Final test accuracy on the image benchmarks. Bars are means, whiskers are ± 1 sd, dots are seeds. Note the truncated y -axes.

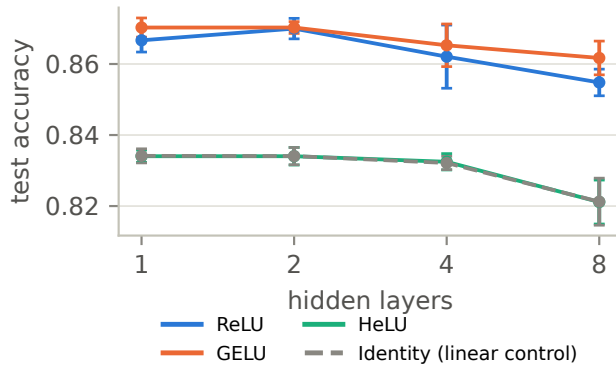


Figure 6: Fashion-MNIST MLP test accuracy versus number of hidden layers (E6), 3 epochs, 3 seeds.

map, and adding HeLU layers does not change that in practice.

4.5 Mechanism: HeLU networks are linear

Two direct measurements explain the results above (Figure 7, Table 5). First, an affine function of the raw pixels explains an R^2 of 1.00000 of the variance in a trained HeLU network’s logits, against 0.957 for ReLU. The HeLU network is linear to within measurement precision. Second, only about 2.81% of hidden pre-activations lie inside the notch after training, and training does not increase this fraction relative to initialisation. Gradient descent has no incentive to move pre-activations into the notch: doing so changes the unit’s output by at most 0.06 and its gradient by 10%, which is too small a lever to be worth the loss incurred on the way. The notch is not so much unhelpful as unused.

4.6 Computational cost

Table 6 shows that HeLU is not cheaper either. Implemented as a masked `where` with two comparisons, its forward-plus-backward kernel is about $6\times$ slower than ReLU’s and $3\times$ slower than GELU’s in our PyTorch CPU measurement. End-to-end epoch time is dominated by the linear layers, so the difference is small in practice, but there is no efficiency argument for HeLU.

Table 4: Depth sweep on Fashion-MNIST (mean $\pm$ sd over 3 seeds).

Hidden layers	ReLU	GELU	HeLU	Linear
1	0.8667 ± 0.0033	0.8703 ± 0.0027	0.8340 ± 0.0017	0.8341 ± 0.0020
2	0.8700 ± 0.0029	0.8703 ± 0.0017	0.8340 ± 0.0025	0.8341 ± 0.0024
4	0.8621 ± 0.0089	0.8653 ± 0.0060	0.8325 ± 0.0023	0.8321 ± 0.0019
8	0.8548 ± 0.0038	0.8617 ± 0.0048	0.8211 ± 0.0062	0.8212 ± 0.0066

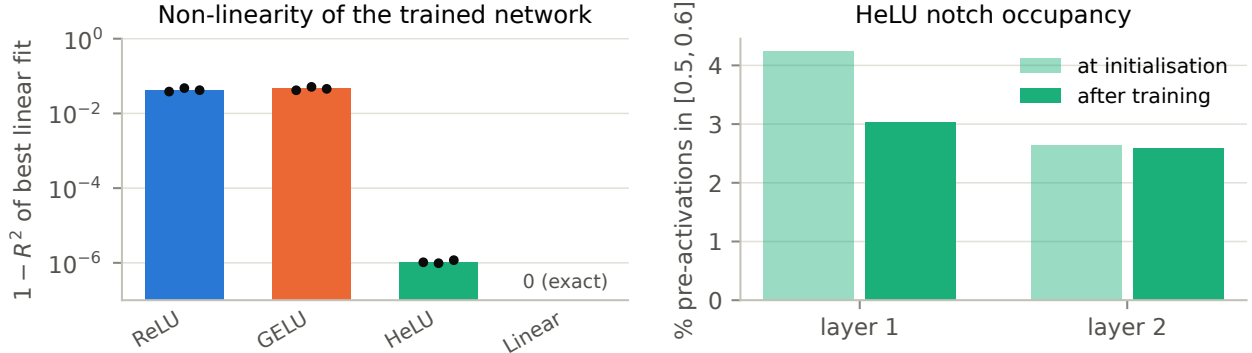


Figure 7: E7. Left: $1 - R^2$ of the best affine fit to each trained network’s logits (log scale; lower means more linear). Right: percentage of HeLU pre-activations that fall inside the notch, per hidden layer, before and after training.

4.7 Variants of the notch

(Variant results pending: run `python run_all.py variants` and `rebuild`.)

5 Discussion

Why HeLU fails. A useful activation function must do two things: give the network a non-linear family of functions to search over, and make that family reachable by gradient descent. HeLU does the first only in the technical sense of Leshno et al. [5], and the second not at all. Its total non-linear content, a 10% dip on a band of width 0.1, is an order of magnitude smaller than the scale of typical pre-activations at initialisation [1], and gradient descent finds nothing to gain by concentrating pre-activations there. The result is a network that is, for all practical purposes, linear, and it performs exactly as a linear network does: adequately on MNIST, where a linear classifier already reaches about 91.5%, and poorly everywhere a curved boundary is required. The one place we saw the notch being used at all, the one-dimensional regression tasks with full-batch training, it produced a jagged staircase of jumps rather than a useful curve.

The CNN caveat. The Fashion-MNIST CNN result deserves emphasis because it is the kind of evidence that could be mistaken for support. HeLU comes within about a point of ReLU on this benchmark only because max-pooling makes the activation function nearly irrelevant, and the identity control demonstrates this by landing at the same accuracy. Any evaluation of a new activation should include a no-activation control for this reason.

Table 5: Linearity of trained Fashion-MNIST MLPs and notch occupancy (means over 3 seeds). The notch-occupancy column for ReLU, GELU, and Identity reports the fraction of *their* pre-activations that happen to fall in $[0.5, 0.6]$ and is included for reference only.

Activation	Test acc.	R^2 of linear fit		% pre-act. in $[0.5, 0.6]$	
		init	trained	init	trained
ReLU	0.8707	0.92093	0.95737	2.65	1.26
GELU	0.8740	0.95841	0.95385	2.41	1.35
HeLU	0.8328	0.99918	1.00000	3.44	2.81
Identity (linear control)	0.8328	1.00000	1.00000	3.45	2.80

Table 6: Cost. Rows 1–3: median wall-clock seconds per training epoch over seeds (4 CPU threads on a shared machine; the median is used because a few runs overlapped with other jobs). Last row: forward plus backward time of the activation kernel alone, measured on an otherwise idle machine.

	ReLU	GELU	HeLU	Linear
MNIST, MLP	1.5	1.6	1.8	1.5
Fashion-MNIST, MLP	1.6	1.6	1.7	1.4
Fashion-MNIST, CNN	12.6	13.5	17.2	11.7
Activation kernel, fwd+bwd, 4M elements (ms)	1.1	2.3	6.8	0.5

What would change the conclusion. We tried the notch as specified. Widening the notch, moving it to straddle zero, or deepening the dip would each increase the available non-linearity and might close the gap, but such a function would no longer be HeLU as defined in (1). We also did not tune learning rates per activation; given that HeLU’s training curves coincide with the linear control’s from the first steps, we do not expect tuning to change the ordering.

Limitations. All experiments are small-scale and CPU-bound: two image datasets of 28×28 greyscale images, networks of at most a few hundred thousand parameters, at most 5 epochs, and 3 to 5 seeds per configuration. The CIFAR-10 download was blocked in our environment. Larger-scale experiments could in principle behave differently, but the mechanism identified in Section 4.5 does not depend on scale: pre-activations avoid the notch because the gradient signal for entering it is negligible, and that is a property of the function, not of the dataset.

6 Conclusion

We set out to verify that HeLU, the identity function with a $0.9\times$ notch on $[0.5, 0.6]$, is a superior activation to ReLU and GELU. Under matched conditions across regression, two-dimensional classification, MNIST, and Fashion-MNIST, it is not. HeLU networks are statistically indistinguishable from purely linear networks and significantly worse than ReLU and GELU networks on every task that requires a non-linear solution. The mechanism is simple: the notch is too small and too rarely visited to function as a non-linearity, so the network stays linear. The only benchmark on which HeLU comes close to the baselines is one in which max-pooling supplies the non-linearity, and a network with no activation at all comes equally close there.

Reproducibility

The code, raw JSON results for every run, figures, and this document are in the `helu-activation-research/` directory of the repository. Running `run_all.py` regenerates the results (about 40 minutes on four CPU cores for the main experiments, plus about 30 minutes for the variants), `make_figures.py` regenerates every figure and table in this paper from them, and `make` in the `paper/` directory rebuilds the PDF.

References

- [1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [2] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.
- [3] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- [4] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [5] Moshe Leshno, Vladimir Ya. Lin, Allan Pinkus, and Shimon Schocken. Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks*, 6(6):861–867, 1993.
- [6] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, 2010.
- [7] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [8] Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for activation functions. In *International Conference on Learning Representations (ICLR) Workshop*, 2018.
- [9] Andrew M. Saxe, James L. McClelland, and Surya Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. In *International Conference on Learning Representations (ICLR)*, 2014.
- [10] B. L. Welch. The generalization of ‘student’s’ problem when several different population variances are involved. *Biometrika*, 34(1–2):28–35, 1947.
- [11] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.